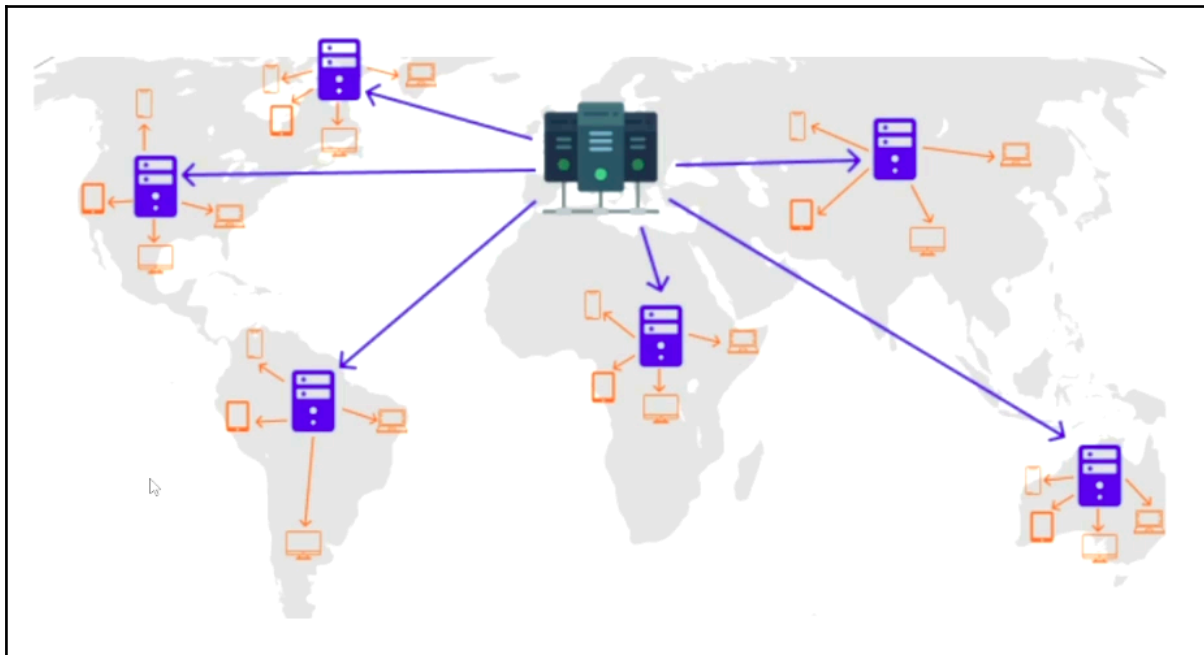


Section 2: System Design Fundamentals

Content Delivery Networks (CDNs)

In this session, we will explore how CDNs enhance performance, reduce latency, and ensure seamless content delivery across the globe. From websites to APIs to video streaming to gaming platforms, CDNs power the modern internet.



Imagine you are trying to load a website hosted on a server halfway across the world. Without CDN, every request must travel that entire distance, leading to slow load times and poor performance. But with a content delivery network or CDN, that experience changes dramatically.

What is a CDN?

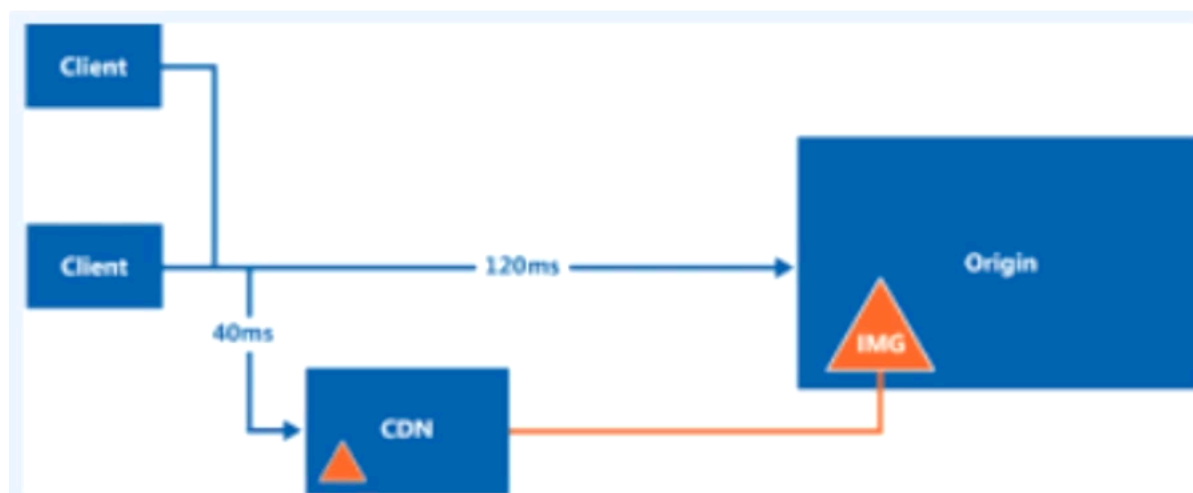
A CDN or content delivery network is a globally distributed network of servers that works together to efficiently deliver content to the user. CDNs strategically place servers around the world to store and serve content closer to the user, and thereby reducing the latency and improving the performance. We need CDNs because:

- It reduces latency. The content is delivered from the nearest CDN node and thus reduces the delays.
- It also enhances the content availability. If one server fails, traffic is rerouted to the next available server.

- It improves load handling. In case any of the CDN servers is overloaded, the CDN will distribute traffic across multiple servers, preventing the overload of a single CDN server or of the origin server.
- It provides security. CDNs provide protections against DDoS attacks, it encrypts traffic, and it prevents malicious access.

Therefore, in modern system design, CDNs are essential for fast, reliable, and secure content delivery, whether it's a website, it's video streaming service, or an API response, CDNs are very critical.

CDN Architecture Overview



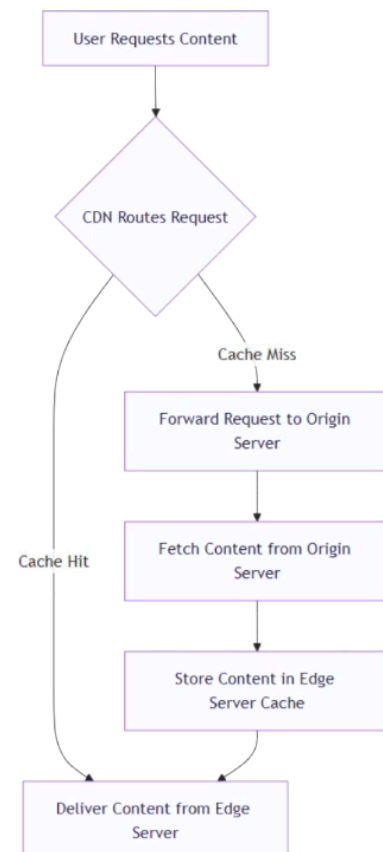
A CDN consists of multiple components that work together to deliver content efficiently while reducing the latency and the server load. There are 3 main components of a CDN network:

- **Origin Server:** This is the source of truth. The origin server is where the original content is stored. It could be hosted in a data center or a cloud provider or on premises also. However, relying on a single origin for ever requests creates performance bottlenecks.
- **Edge Servers (PoPs):** They are also called points of presence. This is a global distribution network. CDNs introduce edge servers, which are strategically placed around the globe to store the cached content closer to the user.
- **Request Routing System:** The entire system is actually configured by a request routing system. When a user makes a request, the request routing system determines the best edge server to serve the content. This decision is based on factors like geographic locations, network latency and the load on the actual server.

How a CDN Works and Handles a User Request

Every time you visit a website, watch a video, or make an API call, your device makes a request for content. Without a CDN, this request travels to the origin server, causing massive delays. You can prevent this using CDNs:

1. The user requests the content (e.g, website video, API data).
2. The CDN directs the request to the nearest edge servers. So when the user makes a request, the domain gets resolved to CDN servers, and the CDN directs the request to the nearest edge server. Instead of sending requests to the distant origin server, the CDN intelligently routes it to the closest edge server. This decision is based on factors like the user's geographic location, network latency, and current server load.
3. If the requested content is already stored on the edge server meaning it has already been cached on the edge server, it is delivered instantly to the user. So it reduces latency and improves the performance.
4. If the content is not available on the edge server, the request is forwarded to the origin server. The origin retrieves the contents, and sends it back, and also caches it on this CDN server for the next upcoming request. This ensures that the future users requesting the same content will at least experience the faster load times.



Therefore, in short a CDN acts as a smart middleman. It ensures that the users get their content faster and with less load on the origin server. This process significantly improves website speed and it reduces the bandwidth usage and enhances the overall user experience.

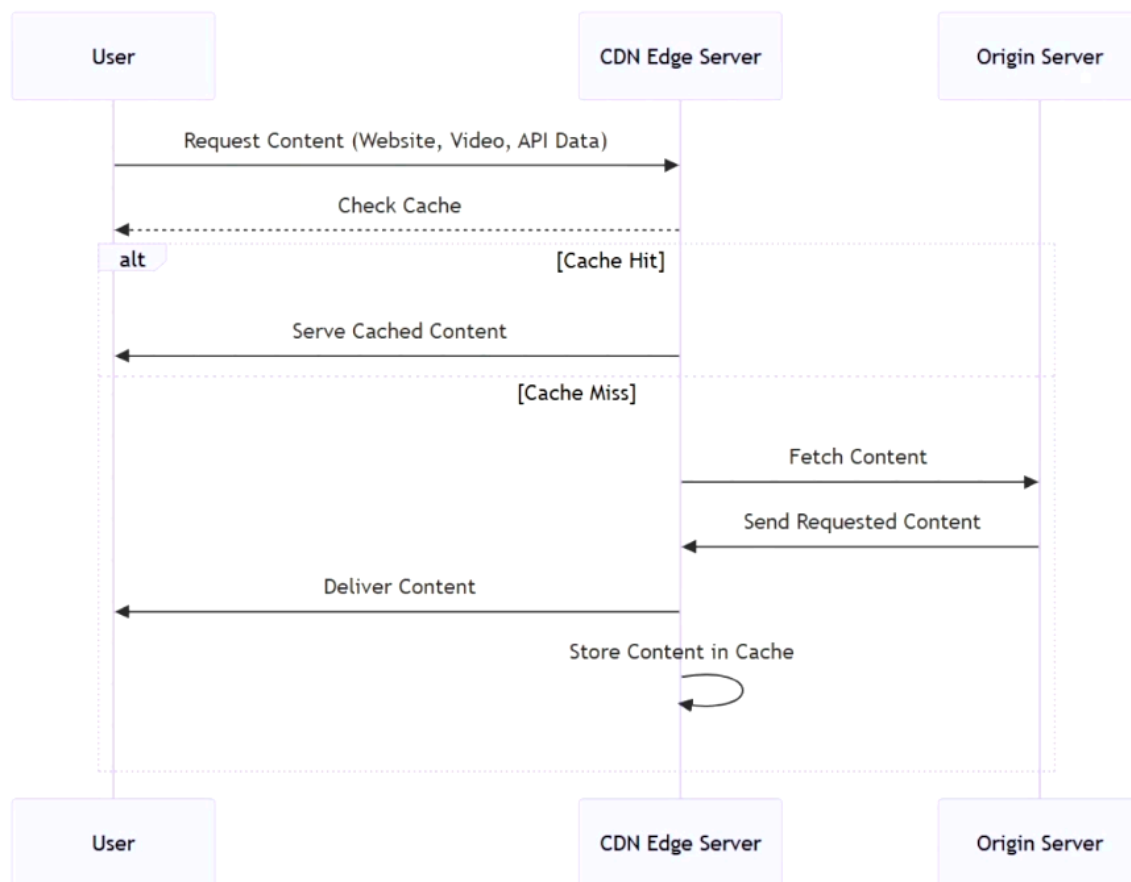
Key CDN Features/Benefits of CDN Working Mechanism

There are many key features which make CDN so powerful. CDNs do much more than just caching the content. They optimize performance, balance traffic, reduce cost, and enhance security.

- **Caching & Replication (Reduce Latency):** This is one of the primary benefits of CDNs, storing frequently accessed content on edge servers closer to the user. This reduces the load on the origin server and it minimizes latency.
- **Load Balancing & Request Routing (Improved Availability):** Traffic is distributed across multiple PoPs to ensure reliability and prevent overload depending on the user's location and the network conditions. CDNs direct traffic to the appropriate server. The strategy could be geo-based routing, latency-based routing, or load-aware routing.
- **Compression & Optimization (Lower Bandwidth Costs):** CDNs can also optimize the content by compressing files and reducing the bandwidth consumption. There are technologies like gzip, shrinking HTML, CSS, and JavaScript files, image optimization, lazy loading, minification, bundling, which CDNs can do, which will actually compress the actual content going to and fro, and then optimize the actual data transfer that is happening across servers.
- **Security Features (Better Security):** Beyond performance, CDNs also provide the essential security features to protect the websites from threats like DDoS protection, SSL encryption, and there are also things like bot mitigation that can happen at the CDNs end.

A well-architected CDN not only speeds up the content delivery, but also ensures reliability, reduces the bandwidth cost, and strengthens security.

Content Caching and Replication



CDNs work by storing copies of frequently accessed content on edge servers, which ensures fast delivery, and it reduces the load on the origin server. But how does a CDN decide what to cache and for how long?

1. First, when a user requests the content for the first time, the CDN fetches it from the origin server and stores a copy on the nearest edge server. This cached copy is then served instantly to the future user. So there could be a hit for the first time user, but the future users will get the benefit of accessing the data from the cache server and thus eliminating the need to repeatedly contact the origin servers.
2. Cache content isn't stored forever, but because the content on the original server could change frequently. So there is a setting called TTL (Time-To-Live), that determines how long an object stays in the cache before it is refreshed. CDNs can define TTL values based on how frequently their content changes. For example, static content can be cached for days or weeks because it does not change. Dynamic content like API responses or personalization data can be cached for a shorter period of time.

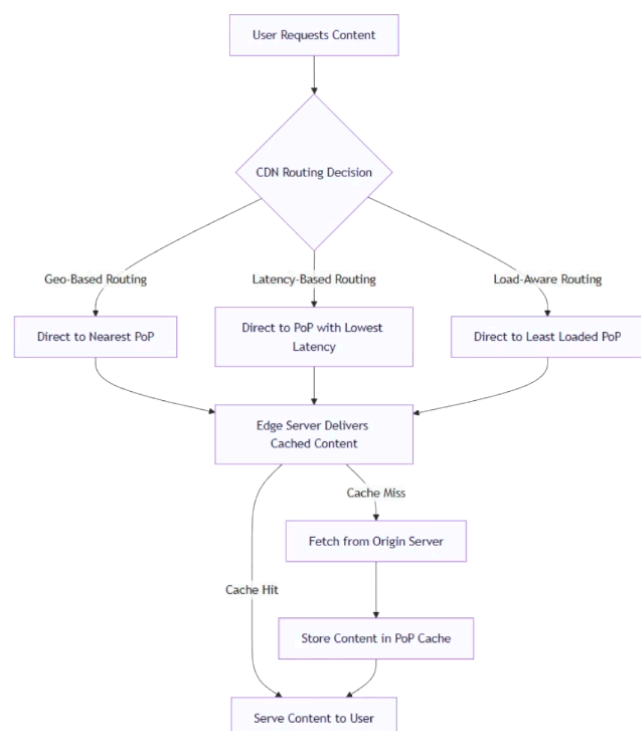
3. Sometimes a content needs to be updated before the TTL expires. So CDNs offer multiple ways of cache invalidation also. So these techniques will ensure the freshness of the cached content without waiting for TTL to expire. There are techniques like manual purge, where the website owner manually removes the outdated content from the cache. There could be things like stale while revalidate. The CDN server will actually serve the expired cached version, but it will also check with the origin server whether there is a new version available. So there could be a hit that will be taken by one user where it serves the stale content, but at the same time it will refetch the content from the server asynchronously, so that the future request will get the fresh content. There is also a technique called cache control headers in which the developers can decide the caching rules in the HTTP headers to get more control on the cache behaviour of CDNs.

So, caching is important and by effectively caching and replicating content, CDNs improve load times, reduce bandwidth usage, and minimize the overload of origin servers.

Load Balancing/Failover Handling and Request Routing

CDNs have intelligent traffic management rules baked inside them. And this is done through load balancing, failover strategies, and request routing strategies.

1. **Load Balancing:** A CDN consists of multiple points of presence worldwide. So instead of overloading a single server, the CDN balances incoming requests across the multiple POPs, ensuring that the smooth performance is there without any failures.
2. **Failover Strategies:** In case there is a server failure, CDN will failover to an alternate server in case it finds that any of that POP is failing. The traffic is seamlessly redirected to the next closest POP.
3. **Request Routing Strategies:** There are multiple request routing strategies:



- *Geo-based routing:* CDNs use geographic routing to direct users to the closest POP, ensuring low latency and faster load times.
- *Latency-based routing:* Rather than just using geography, CDNs also redirect the user to the POP with the fastest response time at this point of time based on the real-time network conditions.

- *Load-aware routing*: To prevent the congestion for a single CDN server, the load-aware routing distributes the request to the least busy POP, ensuring that no single server gets overloaded a lot.

Compression & Minification

One of the key ways CDNs improve performance is by reducing the size of the files before delivering them. Smaller files mean faster load times, lower bandwidth usage, and a smooth user experience. The below are some of the techniques that are used by CDNs for compression and minification:

- Gzip & Brotli compression: Reduces text based file size.
- Image optimization (Webp, AVIF): Optimizes images to faster formats like WEBP.
- Minifying the CSS and JS files sent back to the user.

By applying the above compression techniques, image optimization, and minification technique, CDNs significantly reduces the size and speed up the content delivery, thus improving the user experience and reducing the bandwidth cost.

Security Features

CDNs play a critical role not just in terms of performance but also in terms of security by protecting websites and applications from cyber attacks.

- **DDoS mitigation**: The key security features of CDNs could be DDoS mitigation. There could be a possibility that the CDNs implement rate limiting, it implements traffic filtering, anomaly detection, so that anytime there is a DDoS attack being tried in the origin server, the CDNs will mitigate it.
- **SSL/TLS Offloading**: There is also a possibility of SSL offloading. Everytime we enter sensitive data like password and payment details, that data gets encrypted and this encryption ensures that your information is protected. CDNs enhance security by handling the SSL encryption. This will ensure that the data remains private and safe during the transmission.

So there are things like DDoS protection, traffic filtering and SSL encryption. CDNs will actually provide a lot of security along with the performance optimizations for your application.

Use Cases of CDNs

CDNs are used by almost every major online platform today, from streaming services to e-commerce sites to APIs and web applications. Let's explore some of the most common use cases where CDNs make a huge impact.

1. **Static Content:** Faster loading for common assets like static images, videos, javascript, css, html files. These things don't change that frequently on our servers and these are our static content. CDNs store these at the edge and it will ensure that the ultra fast delivery is happening. An e-commerce site will cache the product images on CDN because the product images will not change that frequently.
2. **Dynamic Content:** The API responses are an example of dynamic content. So optimizing the API responses and personalized pages can be done using CDNs. Dynamic content is very user specific and frequently changing. So what CDN can do is that even though it can't cache all the dynamic content, it can actually optimize it at least by doing compression, minification or bundling. For example: A banking application can actually use CDNs to route API requests through the fastest path. So there could be a possibility that the origin server is very far away, but the CDNs can determine the fastest route to reach there. So even though we have dynamic content being loaded, the intelligent request routing kicks in and it ensures that the shortest path and the fastest route is being taken.
3. **API Acceleration (CDNs for APIs):** APIs power modern applications, but very frequently API requests lead to latency issues and server overload. CDNs help by caching the common API responses at the edge and thus reducing the unnecessary downtrip to the origin. But we need to implement very careful cache invalidation techniques because the data that it is returning from the API could get changed at the origin very frequently.
4. **Edge Computing:** Edge computing is something like processing the data closer to the user rather than processing it on the origin server. So what modern CDNs can do is that they can actually have this feature of edge computing, where lightweight processing can actually happen at the CDN end also rather than going to the origin server. A good example of this is a video streaming platform where it can actually use the edge servers to transcode the videos in real time and thereby optimise the playback on your different devices.

So speeding up static content delivery to optimizing APIs and enabling edge computing, CDNs are essential for modern web applications and we have just looked at some of the use cases of CDNs.

Wrap-Up & Key Takeaways

- CDNs improve performance by caching and distributing content globally.
- They reduce latency, improve load times, and lower bandwidth costs.
- CDNs enhance security with DDoS protection, SSL/TLS encryption, and WAFs.
- Use cases include static content delivery, API acceleration, and edge computing.
- Next Topic:
 - Networking Fundamentals Summary & Best Practices

Interview Questions

- Basics of CDN
 - What is a CDN, and how does it work?
 - Why do we need CDNs in system design?
 - What are the key benefits of using a CDN?
- CDN Architecture & Components
 - Explain the difference between an origin server and an edge server in a CDN.
 - What is a PoP (Point of Presence) in a CDN?
 - How does request routing work in a CDN?
- Caching Strategies & Content Delivery
 - What is a cache hit vs. cache miss, and how does a CDN handle them?
 - Explain cache expiration and TTL (Time-To-Live) in a CDN.
 - What are cache invalidation strategies, and why are they important?
 - How does a CDN handle dynamic content caching?
- Load Balancing & Failover Handling
 - How do CDNs use load balancing to improve reliability?
 - Explain different request routing strategies (Geo-based, Latency-based, Load-aware).
 - What happens if a CDN PoP fails? How does failover handling work?
- CDN Optimization Techniques
 - What compression and minification techniques do CDNs use?
 - How does a CDN optimize image and video delivery?
 - How does API acceleration work in a CDN?
- CDN Security & Challenges
 - How does a CDN protect against DDoS attacks?
 - What is SSL/TLS offloading, and why is it useful?
 - What are some challenges in CDN implementation, and how can they be mitigated?
- Advanced CDN Topics
 - How does a multi-CDN architecture work?
 - What is edge computing, and how does it relate to CDNs?
 - How would you design a CDN for a large-scale video streaming platform?
 - How do CDNs help in real-time applications like online gaming or stock trading?